Safe Edges

2025

Smart Contract Audit REPORT

ASSESSED ON Nov, 2025

The Riglabs

Security Assessment

SAFEEDGES.IN

Moor Toke/Airdrop Draft Audit Report

Table of Contents

Executive Summary

Item	Details

Scope	Review of the following files:
	 `Airdrop-contract-`
	 `MoorToken.sol`

Timeframe	November 2025
Methodology	Comprehensive manual code review, static analysis, threat-modeling, and best-practice compliance verification

Severity Summary	HIGH
Overall Risk	MEDIUM
Status	Under Review

Techniques and Methods

Code quality checks

Best practices & documentation review

Gas efficiency analysis

Reentrancy & ERC compliance checks

Analysis Types

Structural Analysis – Reviewed design patterns & contract structure

Static Analysis – Automated scanning for vulnerabilities

Manual Analysis – Line-by-line code review

Gas Consumption – Optimization analysis

Tools: Remix IDE, Foundry, Slither, Mythril, Solhint

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Open	Resolved
Security vulnerabilities identified that must be resolved and are currently unresolved.	Security vulnerabilities that have been identified and successfully fixed or mitigated.
Acknowledged	**Partially Resolved**
Vulnerabilities which have been acknowledged but are yet to be resolved.	Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

ID	Title	Severity
H-01	Incorrect Token Recipient in `claim` Function`	HIGH
H-02	Incorrect Deadline Validation Using Mismatched Timestamp Formats	HIGH
L-01	Missing `total_allocation` Enforcement For Token Distribution in `claim` Function	LOW

Detailed Findings

[H-01] Incorrect Token Recipient in `claim` Function

Description Within the `claim` function, the contract transfers the airdropped tokens to the caller rather than the specified `recipient`. This contradicts both the intended behavior of the function and the accompanying NatSpec comments, which indicate that the tokens should be sent to the designated `recipient`. As implemented, the `recipient` parameter has no effect on the actual transfer.

```
#[storage(read, write)]
 fn claim(amount: u64, recipient: Identity, tree_index: u64, proof: Vec<b256>) {
// Check if paused
 require(!storage.is_paused.read(), StateError::ContractPaused);
//existing code
// Transfer MOOR tokens to recipient
@>>         transfer(msg_sender_ident, storage.moor_asset.read(), amount);
```

Impact Airdropped tokens are delivered to the caller (`msg_sender`) instead of the intended recipient, rendering the `recipient` argument functionally meaningless and potentially breaking workflows that rely on delegated claiming or custodial recipients.

Recommended mitigation Consider adding these changes:

```
/// Errors related to input validation
pub enum InputError {
/// The amount must be greater than zero
 InvalidAmount: (),
/// The tree index is out of bounds
 InvalidIndex: (),
+    InvalidRecipient: (),
}
```

```
// Mark as claimed
 storage.claims.insert(tree_index, true);
// Transfer MOOR tokens to recipient
-        transfer(msg_sender_ident, storage.moor_asset.read(), amount);
+        require(recipient != Identity::Address(Address::zero()), InputError::In-
validRecipient);
+        transfer(recipient, storage.moor_asset.read(), amount);
```

[H-02] Incorrect Deadline Validation Using Mismatched Timestamp Formats

Description The deadline checks in both `claim` and `clawback` compare a UNIX timestamp against Fuels native block timestamp, `std::block::timestamp()`, which is expressed in TAI64 format. Because `UNIX` and `TAI64` timestamps are not directly comparable, this check is inherently incorrect and results in flawed deadline validation.

```
#[storage(read, write)]
 fn claim(amount: u64, recipient: Identity, tree_index: u64, proof: Vec<b256>) {
// Check if paused
 require(!storage.is_paused.read(), StateError::ContractPaused);
// Check if airdrop has expired
@>>        require(std::block::timestamp() <= storage.end_time.read(), TimingError::Air-
dropExpired);
```

```
#[storage(read)]
 fn clawback(amount: u64, recipient: Identity) {
// Only owner can clawback
```

```
_only_owner();
// Can only clawback after expiry
@>>         require(std::block::timestamp() > storage.end_time.read(), TimingError::Air-
dropNotExpired);
```

Impact Airdrop expiry enforcement may fail entirely or behave unexpectedly, allowing claims after the intended deadline or preventing valid claims before it.

Recommended mitigation We recommend using the Fuel SDK Date conversionutilities to handle timestamp conversions. Alternatively, you may implement the necessary conversions manually, ensuring both TAI64 and UNIX timestamps are properly aligned.

[L-01] Missing `total_allocation` Enforcement For Token Distribution in `claim` Function

Description The contract sets a `total_allocation` value during initialization to define the total number of tokens intended for airdrop distribution. However, this value is never enforced during the lifecycle of the contract. The `claim` function does not track cumulative claimed amounts nor validate that a new amount claim keeps the total distributed tokens within the declared allocation.

As a result, the contracts only safeguard is whether it physically holds enough tokens. If the contract is funded above total_allocation, and if the Merkle tree encodes leaves whose aggregate amount exceeds it, users can successfully claim amounts beyond the intended cap

Impact `total_allocation` provides no real protection or enforcement

Recommended mitigation Consider enforcing the declared `total_allocation` by introducing a total_claimed counter and rejecting any claim that would cause the cumulative distributed amount to exceed the allocation.

Safe Edges

REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US

SAFEEDGES.IN